

Jaypee University of Engineering and Technology, Guna
B.Tech. (Semester II)
Object Oriented Programming Lab (CS202)
Lab Exercise 5

1. Write a **C++ program** to manage **two bank accounts** using a **menu-driven approach** with proper error handling.

Class: BankAccount

Data Members:

- name – Depositor's name
- acc_no – Account number
- acc_type – Type of account (Savings/Current)
- bal – Account balance

Member Functions:

- **initializeAccount()** – Assign initial values.
- **deposit()** – Add an amount to the balance.
- **displayDetails()** – Show depositor's name and balance.
- **Friend Function:** withdraw() – Withdraw money after checking balance.

Main Program Requirements:

- Create **two accounts** and initialize them.
- Implement a **menu-driven system** with options:
 - **Deposit money** – Ask for the account number and amount to deposit in main(), then call deposit().
 - **Withdraw money** – Ask for the account number and amount to withdraw in main(), then call withdraw(). (Use friend function)
 - **Display account details** – Ask for the account number in main(), then call displayDetails().
 - **Exit** – Allow the user to terminate the program.

Error Handling & Validations:

- **Invalid Input Handling:** If the user enters an incorrect **account number**, **negative amount**, or an option that is not in the menu, display an appropriate **error message** and **re-prompt the user**.
- **Insufficient Balance Check:** If the user tries to withdraw an amount greater than the available balance, display "**Insufficient balance! Transaction failed.**"

- **Loop Until Exit:** The program should **continue executing** until the user chooses the **exit option**.
2. Write a **C++ program** to perform **distance addition** using two classes:

Class 1: DM (Distance in Meters & Centimeters)

- **Data Members:**
 - meters (integer)
 - centimeters (float)

Class 2: DB (Distance in Feet & Inches)

- **Data Members:**
 - feet (integer)
 - inches (float)

Operations & Requirements

- **Input Handling (From main())**
 - Define a **function to take input** for DM (meters & centimeters).
 - Define another **function to take input** for DB (feet & inches).
 - Ensure the values entered are **non-negative**.
 - **Do not take any input inside addDistance() function.**
- **Friend Function – addDistance()**
 - Add a DM object and a DB object.
 - The result should be **stored in either DM or DB**, based on the unit preference chosen by the user.
 - **Hint:** Design **two different versions** of addDistance():
 - One that returns the result in **meters & centimeters (DM)**.
 - Another that returns the result in **feet & inches (DB)**.
- **Menu-Driven Program (Loop Until Exit)**
 - **Option 1:** Add two distances and display the result in meters & centimeters.
 - **Option 2:** Add two distances and display the result in feet & inches.
 - **Option 3:** Exit the program.
- **Conversion Hint**
 - 1 meter = 3.28084 feet
 - 1 foot = 0.3048 meters
 - 1 inch = 2.54 centimeters
 - 1 centimeter = 0.393701 inches
- **Error Handling & Validation**
 - **Invalid Input Handling:** If the user enters an invalid menu option, display an error and re-prompt.

- **Negative Distance Handling:** Ensure distances entered are non-negative.
 - **Loop Until Exit:** The program should continue executing until the user chooses the exit option.
3. Write a C++ **program** using **classes** to analyze the annual examination results of **4 students**. The program should provide a **menu-driven interface** to allow the user to perform different operations on student data.

Tasks to Implement:

- **Read and store student data**, including **Roll Number** and **marks in 3 subjects**.
- **Store marks using an array data member** instead of storing them separately for each subject.
- **Calculate the total marks** obtained by each student.
- **Identify the highest marks** in each subject and display the **Roll Number** of the student who secured it.
- **Determine the student who obtained the highest total marks**.

To efficiently manage student data, implement the solution using an **array of objects**. Define a **class Student** that contains **Roll Number and an array data member for storing marks**. Use **member functions** to perform calculations and display results.